

Bachelor Thesis

in the degree program

Informatik

for the attainment of the degree of

Bachelor of Science

Topic:

**Pattern Discovery in Historical Stock Price Time Series
Using Unsupervised Machine Learning**

submitted by:

Mohamed Hamzaoui

Student ID No.: 1462582

At Faculty 2: Computer Science and Engineering
At Frankfurt University of Applied Sciences

First Examiner: Prof. Dr. Martin Simon
Second Examiner: Prof. Dr. Jörg Schäfer

Submission Date: 06.04.2026

Declaration of authenticity:

I, Mohamed Hamzaoui, confirm that this examination is my own work and that I have clearly referenced all tools and sources used. I have not made use of the work of any other student(s). Any sections quoted or paraphrased from other works are clearly marked as such. This examination has not been used for any other examination procedure of the Frankfurt University of Applied Sciences or another university in its entirety or in substantial parts. I understand that a plagiarism check may be carried out.

I used AI-based writing tools and/or other AI applications to write my assignment. I used these tools for guidance only. I confirm that most of the assignment is in my own words. I have listed all AI applications used, including their product name and source (URL), and provided a complete overview of all functions used for the purpose of writing this examination.

Frankfurt am Main, 06.04.2026

Signature:

A handwritten signature in black ink, appearing to be 'M. Hamzaoui', written over a horizontal line.

Abstract

This thesis investigates whether different unsupervised machine learning algorithms can identify recurring price patterns in financial time series data and evaluates the extent to which the resulting structures are well-separated, compact, and consistent across different methods, indicating structure beyond random assignment. The analysis is based on adjusted daily closing prices for S&P 500 constituent stocks covering the period from 2000 to 2025. The time series are segmented into overlapping windows of 50 trading days and each segment is normalized using Min-Max scaling, then three algorithms are applied on the data: K-Means with k determined via the elbow method and silhouette score, Agglomerative Hierarchical Clustering with Ward linkage, and Density-Based Spatial Clustering of Applications (DBSCAN). A qualitative inspection of the cluster centers suggests that K-Means and hierarchical clustering recover some broadly comparable smooth 50-day trajectory forms. However, the correspondence between individual clusters is only approximate, and the two methods do not yield identical partitions of the data.

Cluster quality is assessed using the Silhouette Score, Calinski–Harabasz Index, and Davies–Bouldin Index, while cross-method consistency is evaluated using the Adjusted Rand Index and Normalized Mutual Information. The results show that K-Means outperforms hierarchical clustering on the Silhouette Score and Calinski–Harabasz Index, although both methods yield low silhouette values, indicating weak overall cluster separation.

Cross-algorithm comparison shows a moderate level of agreement between K-Means and Hierarchical Clustering, indicating the presence of non-random underlying structure. In contrast, DBSCAN fails to identify multiple clusters, suggesting the absence of well-separated density regions in the high-dimensional feature space.

Table of Contents

Abstract.....	3
List of Figures.....	6
List of Tables.....	7
List of Abbreviations.....	8
1 Introduction	9
1.1 Background	9
1.2 Motivation and Research Gap.....	9
1.3 Research Questions	10
1.4 Contributions	10
1.5 Thesis Structure	11
2 Related Work	11
2.1 Time Series Clustering	11
2.2 Unsupervised Machine Learning Clustering in Finance	11
2.3 Pattern Discovery in Financial Time Series.....	12
3 Data and Preprocessing.....	12
3.1 Data Source and Scope	12
3.1.1 Data Quality	13
3.1.2 Segmentation	14
3.1.3 Normalization	14
3.1.4 Sampling	15
4 Clustering Algorithms	16
4.1 K-Means	16
4.2 The Elbow Method.....	16
4.3 Agglomerative Hierarchical Clustering	17
4.4 DBSCAN	18
4.5 t-SNE Embedding.....	19
5 Cluster Validation	20
5.1 Internal Validation Metrics	20
5.1.1 Silhouette coefficient	20
5.1.2 Calinski–Harabasz Index	20
5.1.3 Davies-Bouldin Index	21
5.2 Cross-Algorithm Consistency Metrics	21
5.2.1 Adjusted Rand Index	21
5.2.2 Normalized Mutual Information	22
6 Methodology	22

6.1	K-Means	23
6.2	Agglomerative Hierarchical Clustering	24
6.3	DBSCAN	24
6.4	t-SNE Visualisation	25
7	<i>Results and Analysis</i>	26
7.1	K-Means	26
7.2	Agglomerative Hierarchical Clustering	26
7.3	Cluster Center Analysis	28
7.4	DBSCAN	29
7.5	Internal Validation Metrics	30
7.6	Cross-Algorithm Consistency	30
7.7	t-SNE Visualisation	31
8	<i>Discussion</i>	32
8.1	Interpretation of Results	32
8.2	Answering the Research Questions	32
8.3	Limitations	33
8.4	Directions for Future Research	33
9	<i>Conclusion</i>	34
	<i>Declaration of AI Use</i>	35
	<i>References</i>	36
	<i>Appendix A</i>	40

List of Figures

Figure 1: Elbow method.....	17
Figure 2: Sample cut into four clusters in a dendrogram.....	18
Figure 3: DBSCAN Point Types	19
Figure 4: Elbow Method Graph	23
Figure 5: Silhouette Analysis Graph.....	23
Figure 6: DBSCAN k-Distance Graph	25
Figure 7: K-Means cluster distribution	26
Figure 8: Hierarchical - Dendrogram.....	27
Figure 9: Hierarchical Clustering Cluster Distribution.....	27
Figure 10: Cluster centers for K-Means and Hierarchical	28
Figure 11: Average time series of clusters	29
Figure 12: DBSCAN Clustering Distribution.....	29
Figure 13: Internal validation metrics for all algorithms	30
Figure 14 : Cross-algorithm consistency metrics.....	30
Figure 15: t-SNE Embedding - Algorithm Comparison	31

List of Tables

Table 1: raw data structure.....	13
Table 2: metadata structure	14
Table 3: Summary of dataset properties and preprocessing parameters	15
Table 4: Overview of the Metrics	21
Table 5 : Cross-algorithm consistency metrics.....	30

List of Abbreviations

ARI	Adjusted Rand Index
BCSS	Between-Cluster Sum of Squares
GICS	Global Industry Classification Standard
CH	Calinski–Harabasz Index
CSV	Comma-Separated Values
DB	Davies–Bouldin Index
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
HDF5	Hierarchical Data Format version 5
IPO	Initial Public Offering
MI	Mutual Information
MinPts	Minimum Number of Points
NMI	Normalized Mutual Information
PCA	Principal Component Analysis
RI	Rand Index
SOM	Self-Organizing Map
S&P 500	Standard and Poor’s 500
t-SNE	t-Distributed Stochastic Neighbor Embedding
WCSS	Within-Cluster Sum of Squares

1 Introduction

1.1 Background

Financial markets generate large volumes of complex, high-dimensional, and time-dependent data. Much of this information takes the form of time series, that is, sequences of observations recorded over time, such as daily stock prices. Because their values are ordered chronologically, time series contain temporal structure that is not present in ordinary static data. Extracting meaningful structure from such data remains a central challenge in financial data analysis.

A particularly active direction within this field is the study of short-horizon price segments (fixed-length windows extracted from a longer price series) with the aim of discovering whether certain trajectory shapes recur systematically across different stocks, time periods, and market conditions. If such recurring patterns exist and can be identified reliably, they may contribute to a better understanding of short-term price behavior, help characterise the structure of the price-pattern space, and provide a data-driven basis for market analysis without relying on subjective rule-based classifications.

Unsupervised machine learning, and clustering in particular, has emerged as a natural tool for this type of analysis. Clustering algorithms group similar objects together without requiring labelled training data or predefined categories, making them well-suited to the exploratory discovery of structure in large-scale financial datasets. Time series clustering has been applied across a wide range of scientific domains, from gene expression data in biology to energy consumption in engineering, and has attracted growing attention in finance, where it has been used in many applications such as portfolio analysis, risk assessment, market modelling, and anomaly detection, making it a relevant tool for exploratory analysis of complex financial datasets.

A fundamental challenge in unsupervised clustering is that different algorithms rest on different geometric assumptions about what constitutes a cluster. Centroid-based methods such as K-Means assume compact, roughly spherical clusters of similar size. Hierarchical methods build a nested merge structure without imposing a fixed geometry. Density-based methods such as DBSCAN define clusters as regions of high point density separated by sparse space and make no assumption about cluster shape or number. When applied to the same data, these methods may recover different structures, and it is not clear whether any identified patterns reflect genuine properties of the data or are artefacts of a particular algorithm's working mechanism. Cross-algorithm validation offers a principled response to this challenge: patterns that are recovered consistently across fundamentally different algorithms are more likely to represent genuine structure in the data than those found by only a single method.

1.2 Motivation and Research Gap

Despite the growing body of research on machine-learning applications in finance, studies that apply unsupervised clustering specifically to stock-price time series for the purpose of pattern discovery remain comparatively rare. Most existing work either focuses on grouping assets for portfolio construction or using machine learning methods for return prediction. By contrast, relatively few studies examine whether unsupervised clustering can identify recurring structures directly from stock-price time series without imposing prior assumptions about the underlying structure, and even among these, most do not perform any cluster evaluation or they only consider a single algorithm rather than examining whether the recovered structures remain stable across alternative clustering methods. Consequently, the question of whether financial time series contain recurring structural forms that can be identified consistently by different clustering methods has received less attention. This more specific question defines the research space in which the present thesis is positioned.

The present study was partly motivated by Huang, (2023) a practitioner-oriented study that provides a directly relevant precedent, applying K-Means, hierarchical clustering, DBSCAN and Autoencoder + K-Means to 50-day price segments and reporting strong agreement on harmonic or cosine-like trajectory families. While Huang’s study aimed primarily to explore whether such patterns could be found and whether they were related to profitability, the present thesis focuses more explicitly on methodological comparability and cluster validation. In particular, it uses adjusted closing prices, overlapping 50-day windows with a fixed stride, per-segment Min-Max normalization to isolate shape rather than price level, and a shared sample across all algorithms so that cross-method agreement can be evaluated meaningfully. In addition, the analysis is not limited to visual inspection of average patterns but systematically assesses cluster quality and consistency using internal validation metrics and cross-algorithm agreement measures.

1.3 Research Questions

This thesis investigates whether different unsupervised machine learning clustering algorithms can identify recurring price patterns in financial time series data. More specifically, it examines the extent to which the resulting structures are well-separated, compact, and consistent across different methods, thereby providing evidence of non-random underlying structure rather than purely arbitrary groupings.

The analysis is guided by the following research questions:

1. Can different unsupervised clustering algorithms identify recurring price patterns in financial time series data?
2. To what extent do the resulting clusters exhibit internal quality in terms of separation and compactness?
3. To what extent are the recovered structures consistent across different clustering methods?

To answer this research question, the thesis pursues four concrete objectives:

1. To construct a large-scale dataset of normalized 50-day price segments from S&P 500 constituent stocks covering the period 2000–2025, and to extract a shared fixed sample suitable for multi-algorithm comparison
2. To apply three clustering algorithms: K-Means, Agglomerative Hierarchical Clustering with Ward linkage, and DBSCAN to the shared sample under identical preprocessing and evaluation conditions, ensuring that any observed differences in results are attributable to algorithmic mechanisms rather than differences in input data.
3. To analyse and compare the qualitative structure of the resulting cluster centers.
4. To evaluate the quality of the resulting partitions using internal validation metrics: the Silhouette Score, Calinski-Harabasz Index, and Davies-Bouldin Index and to quantify the degree of cross-algorithm agreement using the Adjusted Rand Index and Normalized Mutual Information.

1.4 Contributions

This thesis contributes to the study of unsupervised pattern discovery in financial time series in several ways. First, it provides a comparative analysis of three fundamentally different clustering algorithms, covering partitioning-based, hierarchical, and density-based clustering paradigms, applied to the same shared sample of normalized 50-day stock-price segments under identical preprocessing and evaluation conditions. Second the study goes beyond purely descriptive or visual cluster analysis by combining internal validation metrics with cross-algorithm consistency measures. In this way, it evaluates not only whether clusters can be formed, but also whether the resulting structures are compact, separated, and

consistent across methods. Third, the thesis provides empirical evidence on the geometric nature of the normalized price-segment space. The DBSCAN result, supported by the t-SNE visualization, suggests that the recovered structures are better understood as overlapping forms within a broadly continuous distribution rather than as discrete isolated populations. This indicates that recurring price patterns, if present, are not sharply defined or universally recoverable across methods.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 reviews the relevant literature on time-series clustering, its application in finance, and prior work on pattern discovery in financial time series. Chapter 3 describes the data source and preprocessing pipeline, including data-quality assessment, segmentation, normalization, and sampling. Chapter 4 provides the theoretical background on the clustering algorithms considered in this study, namely K-Means, Agglomerative Hierarchical Clustering, and DBSCAN and introduces t-SNE Embedding. Chapter 5 introduces the validation framework, including internal validation metrics and cross-algorithm consistency measures. Chapter 6 then presents the methodology of the thesis. Chapter 7 reports and analyses the results of the clustering experiments. Chapter 8 discusses the findings, answers the research questions, outlines the limitations of the study, and suggests directions for future research. Finally, Chapter 9 concludes the thesis.

2 Related Work

2.1 Time Series Clustering

Time-series clustering has been applied across a wide range of domains, including biology, medicine, astronomy, climate science, psychology, robotics, and finance (Aghabozorgi et al., 2015).

More generally, it has been shown to provide useful information in many application areas and has attracted sustained attention as an important component of temporal data mining research (Warren Liao, 2005). Liao (2005) provides one of the most frequently cited surveys of time series clustering methods, organizing approaches into three categories based on whether they operate on raw data, extracted features, or fitted models. Aghabozorgi et al. (2015) later expand this perspective and emphasize that time-series clustering depends not only on the clustering algorithm itself, but also on representation, similarity measure, prototype definition, and evaluation strategy. They also note that partitioning methods remain popular because of their computational efficiency, even though they require the number of clusters to be specified in advance. More recently, Javed et al. (2020) show in a large benchmark study that clustering performance varies substantially across datasets and across combinations of algorithms and distance measures, comparing partitional, hierarchical, and density-based methods under Euclidean, dynamic time warping, and shape-based distances and evaluating the results using the adjusted rand index. Together, these studies imply that there is no universally best clustering method for time series, and that empirical comparison on the target application is essential.

2.2 Unsupervised Machine Learning Clustering in Finance

Applications of unsupervised learning in finance appear less prominent than supervised approaches, which continue to dominate areas such as return prediction, classification, and forecasting. In his bibliometric analysis of 392 research documents published between 2000 and 2024, He, (2025) identifies a strong increase in unsupervised-learning research in finance after 2018 and highlights its main areas of application as market modelling and portfolio analysis, risk analysis, and banking and credit assessment. More generally, unsupervised learning is particularly relevant in finance because it can be used to group similar entities, detect anomalies, and uncover latent structure in data where no reliable ground-truth labels exist. One early application of clustering in finance is provided by Nanda et al. (2010), who

examined the use of clustering for portfolio construction by classifying 106 stocks from the Bombay Stock Exchange. Comparing K-Means, self-organizing maps, and fuzzy C-means, the authors report that K-means produces the most compact clusters among the tested methods, suggesting that clustering can support diversification and simplify stock selection. More recently, Aslam (2025) proposes a stock-selection approach that combines K-Means clustering with a risk-managed momentum strategy to identify promising stocks. The author reports that this clustering-based approach outperforms the standard momentum technique and the stock market indices. Clustering has also been applied in other financial settings: for example, Yang et al. (2014) use DBSCAN to identify suspicious financial transactions in an anti-money-laundering context. Overall, these studies show that unsupervised learning has clear financial relevance, but most existing applications focus on stock grouping for portfolio construction, stock screening, or anomaly detection rather than on the discovery recurring patterns in stock price time series.

2.3 Pattern Discovery in Financial Time Series

The use of clustering for pattern discovery in financial time series, as opposed to prediction or portfolio construction, has received comparatively less attention in the literature. One early contribution in this direction is Fu et al. (2001) who applied self-organizing maps (SOMs) to discover patterns in Hong Kong stock-market data. Their method segments the series using a continuous sliding window, compresses each segment using perceptually important points, and then applies the SOM to cluster the resulting representations. While the SOM imposes a topological structure that aids visualisation, the approach differs from the present thesis in its use of a neural-network-based algorithm and a dimensionality reduction step prior to clustering. A more recent and more directly comparable study is Barradas et al. (2023) who apply unsupervised methods to identify conditions in the stock market using data from Yahoo Finance and Google News. Their two-stage methodology first applies principal component analysis (PCA) to reduce dimensionality and then uses K-Means clustering on the transformed feature space. Using the elbow method and the Silhouette Score, they identify four market conditions, including optimistic, pessimistic, steady, and high-volatility states, with the high-volatility condition representing the smallest share of the data. This study is particularly relevant because it also uses unsupervised learning to uncover latent structure in financial data, however, it differs from the present thesis in that it combines price data with sentiment and macro-financial information and focuses on market conditions rather than on normalized rolling stock-price segments.

Overall, the literature suggests that while clustering has been widely used in finance, most applications focus on grouping assets, supporting portfolio decisions, or detecting anomalies. In contrast, comparatively fewer studies investigate whether unsupervised learning can uncover recurring structures directly from stock-price time series. Furthermore, the extent to which different clustering algorithms recover consistent structures from the same stock-price data has received limited attention, as has the systematic evaluation of cluster compactness and separation. The present thesis is positioned within this more specific gap.

3 Data and Preprocessing

3.1 Data Source and Scope

The data used in this thesis consists of historical adjusted daily closing prices for 499 of the S&P 500 (Standard and Poor's 500) constituent stocks. The price data are retrieved using `yfinance`, a Python library that provides programmatic access to Yahoo Finance's historical data. The ticker symbols of S&P 500 constituents are loaded from a local CSV file (`data/raw/sp500_constituents.csv`). For each ticker,

the adjusted closing price is downloaded. The raw data is then stored in csv format at data/raw/sp500_prices.csv.

The dataset spans the period from January 1, 2000, to December 31, 2025, and is recorded at a daily frequency based on trading days. The resulting dataset is structured as a panel of aligned time series, where each column represents an individual stock and each row corresponds to a trading day.

The S&P 500 index, which tracks the performance of the 500 leading companies in the USA across all 11 GICS sectors, gives a very good heterogeneity and provides enough data points to support meaningful cluster analysis and it has been also the benchmark dataset in prior related work.

The use of adjusted closing prices rather than raw closing prices is an important methodological choice. While the raw closing price is the exact last price of a stock at the end of a trading day, the Adjusted close is the closing price after adjustments for all applicable splits and dividend distributions. For example, if a company with a share price of \$200 executes a 2-for-1 split, the raw closing price drops to \$100 the following day which results in a sudden apparent price halving which does not represent any loss of economic value. This can be misleading when conducting an analysis of the stock price data. The adjusted closing price corrects for this by retroactively scaling past prices. As a result, the time series becomes continuous and reflects true economic returns rather than artificial jumps caused by corporate actions.

Table 1 shows the structure of the raw data:

Date	MMM	AOS		ZBH	ZTS
2000-01-03	19.25739288330078	2.2540669441223145			
2000-01-04	18.492206573486328	2.221588134765625			
⋮	⋮	⋮		⋮	⋮
2025-12-30	160.45004272460938	67.4067153930664		90.79000091552734	125.87252044677734
2025-12-31	159.38482666015625	66.55094146728516		89.91999816894531	125.28502655029297

Table 1: raw data structure

3.1.1 Data Quality

A data-quality check was conducted on the downloaded adjusted closing price series before segmentation. Excluding missing values caused by normal IPO and delisting gaps, no invalid prices less than or equal to zero were found and no duplicate dates were detected, indicating that the dataset is structurally clean. Out of the 499 stocks, 347 have a complete 25-year history, while the remaining series are shorter because some firms entered or left the index during the observation period. These missing values are therefore not treated as data errors but as expected consequences of changing index membership over time. Stocks are not discarded solely for having incomplete histories; instead, any stock with at least 50 consecutive trading days of valid data is retained and allowed to contribute segments to the analysis.

3.1.2 Segmentation

A common strategy for analyzing long financial time series is the sliding window (or moving window) approach. Given a price series of length T , a window of length L is applied with a step (stride) of s to produce a collection of overlapping segments:

$$\text{Segment}_i = \{x_{i,s}, x_{i,s+1}, \dots, x_{i,s+L-1}\}$$

Where x denotes the adjusted closing price of a stock at trading day $i.s$.

In this thesis, the price time series were divided into segments of 50 consecutive trading days ($L = 50$) with a stride of 5 days ($s = 5$) corresponding to one trading week and producing a 90% overlap.

The window of 50 trading days corresponds to approximately two and a half calendar months of trading activity. This length is long enough to capture a meaningful price trend or reversal, while remaining short enough to stay within a single market phase. This choice is consistent with the segment lengths used in Huang (2023).

The total segmentation procedure produced 570,321 segments across all 499 stocks. Each segment is associated with metadata consisting of the ticker symbol and the start day index. The metadata is stored in `data/processed/segments_metadata.csv`. The raw price data are stored in `data/processed/segments.h5` using the HDF5 format. Table 2 shows the structure of the metadata:

segment_idx	ticker	start_day
0	MMM	0
1	MMM	5
⋮	⋮	⋮
570316	ZTS	3175
570317	ZTS	3180

Table 2: metadata structure

3.1.3 Normalization

After segmentation, each segment was normalized independently using Min-Max normalization method. Normalization ensures that clusters are formed based on the patterns within a window rather than on the absolute price level.

Two segments that both rise steadily by 10% over 50 days exhibit identical patterns. Without normalization, if one belongs to a \$10 stock rising from \$10 to \$11 and the other to a \$500 stock rising from \$500 to \$550, they would be in very different regions of the raw feature space and will be assigned to different clusters. Per-segment normalization places both segments in the same region, allowing the clustering algorithm to identify their structural similarity.

The Min-Max normalization formula for a segment s_i containing values $(x_1, x_2, \dots, x_{50})$ is:

$$\tilde{x}_t = \frac{x_t - x_{\min}}{x_{\max} - x_{\min}}$$

where:

- x_t is the price at day t within the segment
- $x_{\min} = \min(x_1, x_2, \dots, x_{50})$ is the minimum price within that segment
- $x_{\max} = \max(x_1, x_2, \dots, x_{50})$ is the maximum price within that segment
- $\tilde{x}_t$ is the normalized value for day t , and it lies in $[0, 1]$

This maps each value of the segment to the unit interval $[0, 1]$, removing absolute price level while preserving the shape (the relative trajectory of prices) over the 50-day window.

Flat segments (where $x_{\max} = x_{\min}$) are assigned a constant value of 0.5 to avoid division by zero.

3.1.4 Sampling

The full segment dataset is large enough to make some clustering algorithms computationally infeasible, particularly for the silhouette score, DBSCAN and hierarchical clustering as these methods involve notable computational costs. A random sample of 50,000 segments is therefore drawn without replacement from the normalized dataset and used for the clustering and evaluation steps.

Table 3 summarizes the properties of the data:

Property	Value
Source	Yahoo Finance (yfinance)
Universe	499 of S&P 500 constituents
Price type	Adjusted closing price
Segment length	50 trading days
Stride	5 days (90% overlap)
Total segments	570,321
Normalisation	Min-Max per segment $\rightarrow [0, 1]$
Shared sample	50,000 segments (seed = 42)
Storage format	HDF5

Table 3: Summary of dataset properties and preprocessing parameters

4 Clustering Algorithms

This section introduces the clustering algorithms used in the analysis. The selected methods represent three different clustering paradigms: a partitioning/centroid-based method (K-Means), which assigns observations to clusters based on proximity to cluster centroids; a hierarchical method (Agglomerative Hierarchical Clustering), which builds a nested grouping by iteratively merging similar observations; and a density-based method (DBSCAN), which defines clusters as dense regions separated by areas of lower point density. Including algorithms with different underlying assumptions makes it possible to examine whether recurring structures in the normalized stock-price segments are robust across alternative notions of what constitutes a cluster.

4.1 K-Means

K-Means is the most widely used clustering algorithm. It begins by selecting k data points at random as initial cluster centers. It then performs two phases repeatedly: first, each data point is assigned to the cluster with the nearest center; second, the cluster centers are updated as the mean of all points assigned to each cluster (Shirkhorshidi et al., 2014). The iteration repeats the two steps until the centroids do not change between two consecutive rounds. This is known as Lloyd iteration procedure (Zhao et al., 2018).

Given n data points and a target number of clusters k , the partitioning of the data is done by minimizing the within-cluster sum of squares (WCSS), also called inertia:

$$WCSS = \sum_{j=1}^k \sum_{x_i \in C_j}^n ||(x_i - \mu_j)||^2$$

where C_j is the set of points assigned to cluster j , x_i is a data point, and μ_j is the centroid of cluster j , computed as the mean of all points in C_j .

Despite being widely used, k-Means has several known weaknesses. Ashabi et al. (2021) reviewed 114 studies and found four main issues: the need to pre-define the number of clusters k , the algorithm is highly sensitive to the random initialization of centroids, it is susceptible to noise and outliers and it exhibits poor scalability when applied to large datasets.

This work addresses the pre-defining of k by determining it through the elbow method, Initialization sensitivity is mitigated by running the algorithm 10 times with independent random seeds using k-Means++ seeding, retaining only the run with the lowest inertia. Scalability is managed by operating on a random sample of 50,000 segments rather than the full dataset. the per-segment Min-Max normalization applied in this work addresses the scale issue of the outlier sensitivity.

4.2 The Elbow Method

The elbow method is a widely used heuristic method to determine the number of clusters k in K-Means. This method is based on the idea that we choose a number of clusters so that adding another cluster doesn't give much better modelling of the data. (Bholowalia & Kumar, 2014). It works by running K-means for different values of k (e.g., $k = 1, 2, 3, \dots, 10$) and computing the within-cluster sum of squares (WCSS) for each case. As k increases, WCSS always decreases, because more clusters naturally reduce the distance between points and their assigned centroids.

However, this improvement is not uniform. Initially, increasing k leads to a sharp decrease in WCSS, since large, poorly defined clusters are split into more meaningful ones. After a certain point, the reduction becomes much smaller, meaning that adding more clusters provides only marginal improvement. The optimal value of k for the dataset is typically identified at the “elbow” point in the plot of WCSS (inertia) against k , where the rate of decrease in inertia starts to slow down significantly (Aslam, 2025).

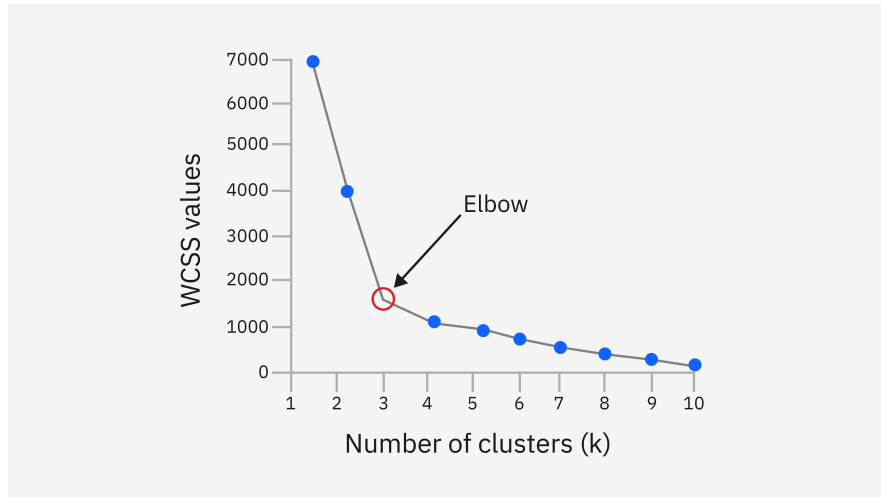


Figure 1: Elbow method. (Winland & Kavlakoglu, 2024)

4.3 Agglomerative Hierarchical Clustering

Hierarchical clustering is a method that builds a nested structure of clusters by successively grouping data points based on their similarity. There are two approaches to hierarchical clustering: divisive and agglomerative clustering. The divisive clustering is a "top-down" hierarchical method, starting in a single cluster and splitting big clusters into small ones.

In the agglomerative approach, which is the most commonly used, the clustering is built from the bottom up. Initially, every point is its own cluster, so that there are n clusters. At each step, the two most similar clusters are merged (Hennig et al., 2016). This process continues until a single cluster contains all points, producing a binary tree called a dendrogram.

In comparison with partitional clustering algorithms such as K-means, hierarchical approaches generally incur a higher computational cost. However, an important advantage of hierarchical methods is that they do not require specifying the number of clusters k in advance, which makes it more suitable for real-world data where finding a suitable k can be tricky (Bouguettaya et al., 2015). Instead, the final number of clusters is determined by cutting the resulting dendrogram at an appropriate level.

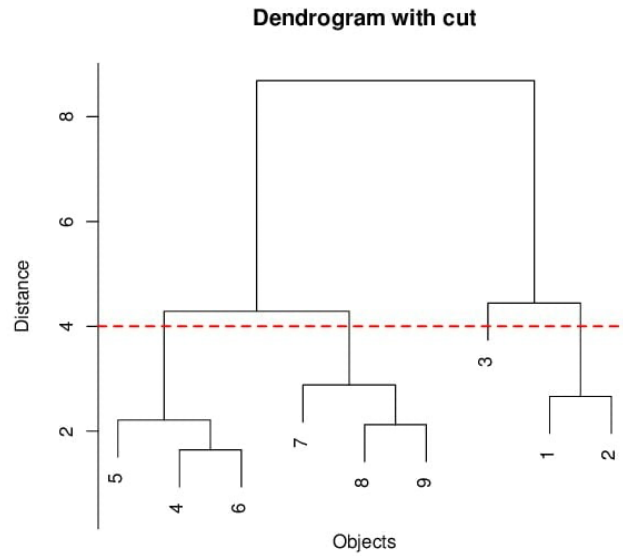


Figure 2: Sample cut into four clusters in a dendrogram (Pereira & Mello, 2013)

The most important component of hierarchical clustering is the linkage criterion. It determines how the distance between two clusters A and B is measured during the merging process. The most common approaches include single linkage, which considers the distance between the closest pair of points of the two clusters; complete linkage, which uses the distance between the farthest pair; average linkage, which takes the mean of all pairwise distances of all objects of clusters; and Ward linkage which is used in the present thesis.

In Ward clustering method, at every step, every possible merge of two clusters is considered (Vijaya et al., 2019). The pair of clusters whose union produces the smallest increase in the value of the sum-of-squares variance are then combined together (Warren Liao, 2005).

Ward linkage minimizes the increase in total within-cluster variance at each merge, which is mathematically equivalent to minimizing the within-cluster sum of squares (WCSS), the same criterion K-Means optimizes. Which means any differences in their results can more plausibly be attributed to the algorithmic mechanism (the merge hierarchy versus iterative centroid optimization) rather than to differences in what the two methods are optimizing.

4.4 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) (Ester et al., 1996) takes a fundamentally different approach. Unlike K-Means and hierarchical clustering, which assign every data point to fixed number of clusters, DBSCAN identifies regions of high point density and treats low-density regions as noise (outliers), which are not assigned to any cluster (He, 2025).

The algorithm operates on two main parameters:

- **ϵ (eps):** the radius that defines the maximum distance between two points to be considered neighbors.
- **MinPts:** The minimum number of neighbors a point must have within distance ϵ to be considered a core point

Based on these parameters, three types of points are identified:

- **Core point:** has at least $MinPts$ points within its ϵ -neighborhood
- **Border point:** lies within the ϵ -neighborhood of a core point but does not itself satisfy the $MinPts$ requirement.
- **Noise point (label = -1):** is neither a core nor a border point and is not assigned to any cluster.

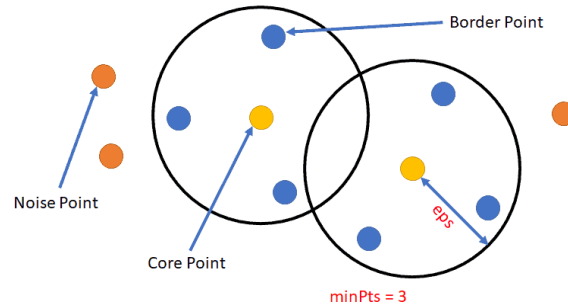


Figure 3: DBSCAN Point Types (Gour, 2023)

DBSCAN forms clusters by starting from an unassigned core point and creating a new cluster around it. The algorithm then expands this cluster by iteratively including all points within the ϵ -neighborhood of the core point to the cluster. If neighboring points are also core points, their neighborhoods are explored recursively, allowing the cluster to grow by connecting all density-reachable points. Border points are absorbed into the cluster of the core point they are nearest to. Noise points receive label -1 .

As a result, DBSCAN can discover any shape class in the dataset (Deng, 2020) and automatically determines the number of clusters. The Algorithm runs in $O(n \cdot \log(n))$ for the average case and $O(n^2)$ for the worst case. Its main limitation is its sensitivity to the parameter choice: a poorly chosen ϵ may cause all points to be in one cluster or all points to be noise.

Unlike K-Means and Hierarchical Clustering, DBSCAN does not require k to be specified, it discovers both the number of clusters and the identification of noise points directly from the data. Furthermore, the algorithm's geometric assumptions differ fundamentally from those of K-Means and Hierarchical Clustering. DBSCAN makes no assumptions about cluster shape and explicitly models the possibility that some points do not belong to any cluster. This makes it a fundamentally different type of comparison: if DBSCAN confirms the same four-cluster structure found by the partition-based and hierarchical algorithm, this would provide strong evidence for the existence of those clusters.

4.5 t-SNE Embedding

t-SNE (t-distributed Stochastic Neighbor Embedding) is a non-linear dimensionality-reduction technique that projects high-dimensional data into a low-dimensional space, typically two dimensions, for visualization purposes. Its main objective is to preserve local neighborhood structure, so that points that are close in the original feature space tend to remain close in the embedding. In this thesis, t-SNE is used as a complementary visualisation tool to provide a two-dimensional perspective on the cluster structure produced by the three algorithms and does not influence the clustering itself.

5 Cluster Validation

This chapter introduces the validation framework used in this thesis. Since the study compares several unsupervised clustering algorithms without access to external ground-truth labels, the quality of the resulting partitions must be assessed indirectly. To this end, the chapter presents internal validation metrics for cluster quality as well as cross-algorithm consistency measures for comparing the structures recovered by different methods.

5.1 Internal Validation Metrics

Clustering validation can be performed using either external or internal evaluation metrics. When an external ground truth is available, clustering quality can be assessed by comparing the obtained clusters to the known labels using external validation measures. However, in many real-world applications, particularly in unsupervised learning, such ground truth is not available, which is also the case in this thesis. In these cases, internal validation metrics are used, which evaluate the quality of clustering and produce an outcome just considering the internal data points of the clusters (Chicco et al., 2025).

Chicco et al. (2025) compare six internal clustering metrics (the Silhouette coefficient, Davies-Bouldin index, Calinski-Harabasz index, Dunn index, Shannon entropy, and Gap statistic) and conclude that the Silhouette coefficient and the Davies-Bouldin index are the most informative for internal evaluation of unsupervised clustering, noting that these two metrics have also been the most widely adopted in the literature. This work employs the Silhouette coefficient, Davies-Bouldin index, and Calinski-Harabasz index to evaluate clustering performance.

5.1.1 Silhouette coefficient

The Silhouette Score (Rousseeuw, 1987) operates at the level of individual points. It measures how well each data point fits within its assigned cluster, yielding a value in the range $[-1, 1]$. A score close to 1 indicates that the point is well matched to its assigned cluster and clearly separated from others; a score near 0 indicates a boundary point; a negative score suggests potential misassignment.

For a given data point i , the silhouette score is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max \{a(i), b(i)\}}$$

where:

- **Intra-cluster distance (a_i):** the average distance between the point and all other points within the same cluster. Lower values indicate that the point is well matched to its cluster. (*What Is Silhouette Score*, 2026)
- **Nearest-cluster distance (b_i):** the average distance between the point and all points in the closest neighboring cluster. Higher values indicate that the point is well separated from other clusters (*What Is Silhouette Score*, 2026).

5.1.2 Calinski-Harabasz Index

The Calinski-Harabasz (CH) Index (Caliński & Harabasz, 1974), also known as Variance ratio criterion, measures the ratio of between-cluster separation (BCSS) to within-cluster dispersion (WCSS) (Chicco et al., 2025). The CH index can take any value greater than or equal to 0. A high CH means better clustering since observations in each cluster are closer together (denser), while clusters themselves are further away from each other (well separated).

$$CH = \frac{BCSS/(k-1)}{WCSS/(n-k)}$$

where:

- BCSS (between-cluster dispersion matrix) is the distance between the centroid of each cluster c_i and the general centroid c .
- WCSS (within-cluster dispersion matrix) is the Euclidean distance between the x data points and the centroid of their cluster c_i .
- k is the number of clusters, and n is the number of data points.

5.1.3 Davies-Bouldin Index

The Davies-Bouldin (DB) Index (Davies & Bouldin, 1979) measures the average similarity between each cluster and its most similar one, taking into account both cluster spread and the distance between cluster centers. The DB index can take any value greater than or equal to 0. Unlike the previous metrics, lower values are preferred, as they indicate that clusters are compact and clearly separated, with minimal overlap. DB of k number of clusters is :

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right)$$

where for each cluster i : S_i represents the average distance of all points in cluster i to its centroid (a measure of intra-cluster scatter), and M_{ij} the Euclidean distance between centroids of clusters i and j .

Table 4 shows an overview of the three evaluation metrics:

Metric	Interval	Meaning
Silhouette coefficient	$[-1; +1]$	The higher, the better
Calinski-Harabasz index	$[0; \infty)$	The higher, the better
Davies-Bouldin index	$[0; \infty)$	The lower, the better

Table 4: Overview of the Metrics (Chicco et al., 2025)

5.2 Cross-Algorithm Consistency Metrics

While internal validation metrics evaluate each algorithm in isolation, cross-algorithm consistency metrics measure the degree to which two independent clustering algorithms agree on the assignment of data points to clusters.

5.2.1 Adjusted Rand Index

The traditional Rand Index is the simplest measure of agreement between two clusterings. Given two clusters A and B over the same n points, it evaluates agreement based on how many pairs of points the two clusterings agree, either both assigned to the same cluster, or both assigned to different clusters:

$$RI = \frac{a + b}{\binom{n}{2}}$$

where a is the number of pairs assigned to the same cluster by both A and B , and b is the number of pairs assigned to different clusters by both.

The Rand Index (Rand, 1971) ranges from 0 to 1, However, it doesn't consider the possibility of chance agreements between the two clusters. This makes the raw Rand Index unsuitable for meaningful comparison.

The Adjusted Rand Index (ARI) (Hubert & Arabie, 1985) overcomes this limitation by adjusting for chance, effectively correcting for agreements that could occur randomly. Its values range from -1 to 1 , where 1 indicates perfect agreement, 0 corresponds to agreement no better than random, and negative values indicate worse agreement than expected by chance.

5.2.2 Normalized Mutual Information

In the Probability theory and Information theory Mutual Information (MI) is a measure that quantifies the amount of information obtained about one random variable by observing the other random variable. Applied to two clusterings A and B , it quantifies how much knowing the cluster assignment of a point under A reduces uncertainty about its assignment under B .

Mutual Information is not bounded and depends on the number of clusters, making it difficult to interpret across different clusterings. Normalized Mutual Information (Strehl & Ghosh, 2002) addresses this by dividing the MI by a normalization factor based on the entropies of the two label distributions (which reflects how much information could theoretically have been shared given the structure of the two clusterings). The result is expressed as a proportion between 0 and 1 , making it directly interpretable regardless of the number of clusters or the size of the dataset. A value of 0 means the two clusterings share no information at all (knowing one tells you nothing about the other). A value of 1 means the two clusterings are identical (knowing one tells you everything about the other). Unlike ARI, NMI is less sensitive to imbalanced cluster sizes.

6 Methodology

The goal of this thesis is to discover recurring price patterns in S&P 500 stock price segments by applying three clustering algorithms and evaluating they converge on consistent groupings. The methodology is designed to make the three algorithms directly comparable: all of them operate on the exact same preprocessed dataset, use Euclidean distance as the similarity measure, and are evaluated using the same set of metrics. This ensures that any differences observed in the results can be attributed to the algorithmic mechanisms rather than to differences in the input data or evaluation procedure.

As described in section 3.1.4, a random sample of 50,000 segments is drawn from the full dataset. The same sample is used across all clustering algorithms. Another reason for using a shared sample is that cross-algorithm consistency metrics, such as ARI and NMI, compare label arrays element by element. Specifically, a position i in the K-Means label array must correspond to the same segment as position i in the hierarchical and DBSCAN label arrays. If the algorithms operated on independently drawn samples, this element-wise correspondence would be lost, and the consistency metrics results would be meaningless.

the source code used for data collection, preprocessing, clustering, evaluation, and visualization is provided in the GitHub repository listed in Appendix A.

6.1 K-Means

As K-Means requires the number of clusters k to be specified in advance, the elbow method is applied to determine an appropriate value from the data. A silhouette score analysis is also implemented to better evaluate the choice of k .

The elbow method is run by computing the total within-cluster inertia for each k and identifying the point of maximum curvature (Elbow), detected automatically as the maximum of the second derivative of the inertia curve. The following graph shows the result of the elbow algorithm:

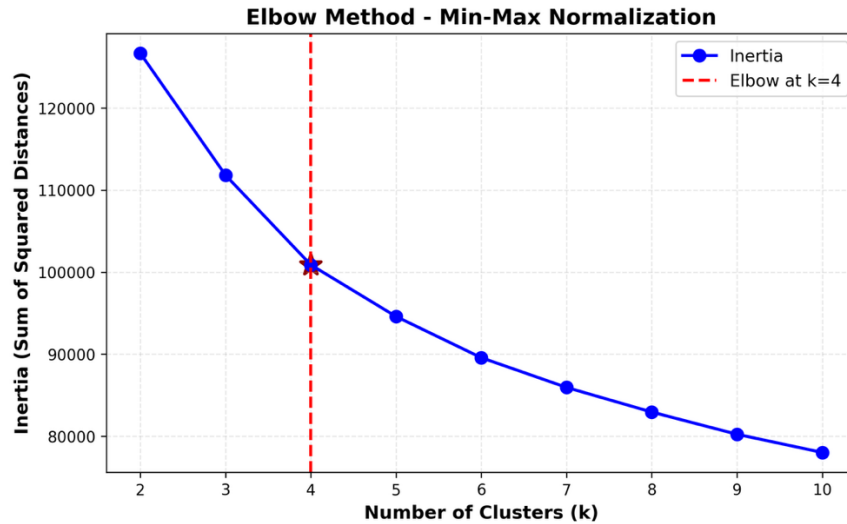


Figure 4: Elbow Method Graph

The Silhouette Score can also be used to find the optimal number of clusters in K-Means. The analysis plots the mean silhouette coefficient for k values from 2 to 10. Higher scores indicate better-separated clusters, which means better choice of k . Following graph represent the result of the implemented algorithm:

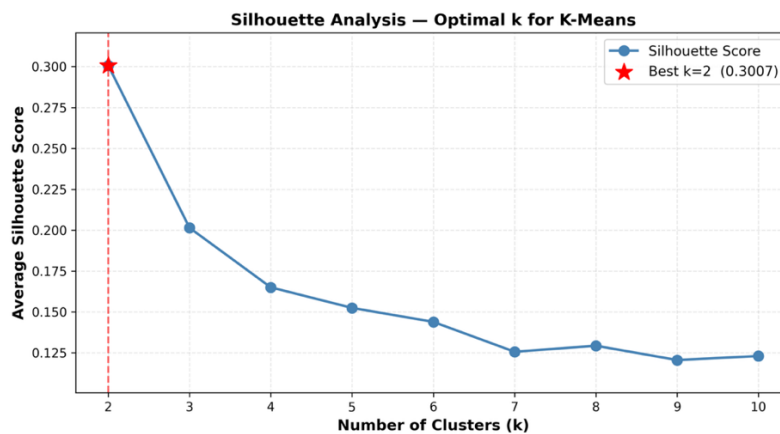


Figure 5: Silhouette Analysis Graph

The elbow method identified $k=4$ as the point of maximum curvature in the inertia curve. However, the curve decreases continuously without a sharp inflection, indicating no strongly dominant cluster count. Similarly, the Silhouette Score decreases monotonically from $k=2$ to $k=10$ with no peak or inflection, indicating the absence of strongly separated cluster structure at any k . The maximum silhouette of 0.307 at $k=2$ is itself not high, confirming that no value of k produces strongly separated clusters in this data.

In the absence of a strong quantitative signal from either metric, $k=4$ is selected on the following grounds. The largest single reduction in marginal inertia gain occurs at the $k=3$ to $k=4$ transition which represents the strongest available elbow signal. Furthermore, $k=4$ is consistent with the four harmonic pattern families reported by Huang (2023) and the four market conditions independently identified by Barradas et al. (2023), providing a basis for cross-study comparison.

K-means clustering is implemented in `src/clustering/kmeans/kmeans.py` using the K-Means class from the scikit-learn library. The model is configured with $k = 4$, `random_state = 42` to ensure reproducibility, `k-means++` initialization, and `n_init = 10`, corresponding to ten independent random initializations, as described in 4.1. The algorithm is then applied to the 50,000 normalized segments.

6.2 Agglomerative Hierarchical Clustering

Agglomerative Hierarchical Clustering is applied to the same 50,000 segment sample using Ward linkage and Euclidean distance. While the dendrogram produced by hierarchical clustering encodes all possible partitions from $k=1$ to $k=n$, a specific cut must be chosen for the final flat partition and for comparison with K-Means. Based on the results of k selection in K-Means the dendrogram is cut at $k=4$ using `scipy's fcluster` function producing four labels which can be compared with the K-Means output.

Hierarchical Clustering is implemented in `src/clustering/hierarchical/agglomerative.py` using `scipy.cluster.hierarchy` for the dendrogram construction and `sklearn.cluster.AgglomerativeClustering` for the final flat clustering.

6.3 DBSCAN

As discussed in section 4.4 DBSCAN's behavior is governed by two parameters: the neighborhood radius, and `MinPts`, the minimum number of neighbors a core point must have within `eps`. Following the standard rule-of-thumb for high-dimensional data (Sander et al., 1998), `MinPts` is set to $2 \times d = 2 \times 50 = 100$, where $d = 50$ is the segment dimensionality. A second, less restrictive value (`MinPts = 50`) was then included in the subsequent grid search as an exploratory alternative. The `eps` parameter is then estimated using a combination of a k -distance graph and a grid search.

The initial `eps` value is estimated using the k -distance graph method (Ester et al., 1996) using $k = 100$. For each of the 50,000 points, the distance to its 100th nearest neighbor is computed using a `NearestNeighbors`. These distances are then sorted in ascending order and plotted. The intuition is that points in dense regions will have relatively small k -distances, whereas points in sparse regions or noise will have larger ones. The point at which the sorted curve bends sharply upward can therefore be interpreted as a natural threshold separating dense regions from sparse space. In the implementation, this elbow is identified automatically as the point of maximum second derivative computed on a downsampled version of 1,000 evenly spaced values for efficiency, yielding an initial estimate of `eps = 1.7515`. This value was treated as a heuristic starting point rather than as a fixed final parameter, since the elbow of the k -distance graph provides only an approximate indication of where dense regions begin to separate from sparse space. Figure 6 shows the k -distance graph:

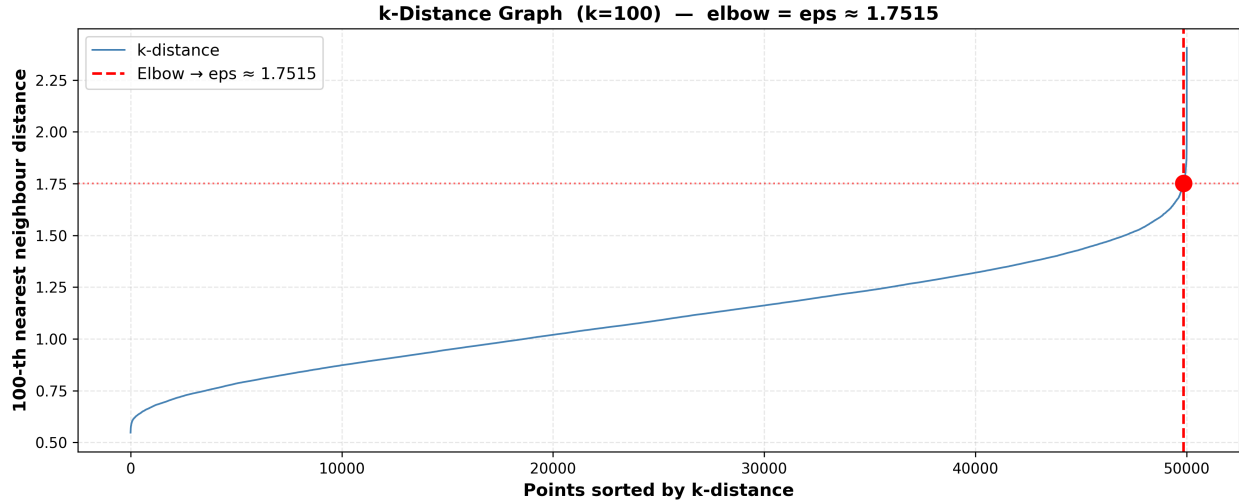


Figure 6: DBSCAN k-Distance Graph

Grid Search for Final Parameters:

Because the k-distance elbow provides only an initial estimate and DBSCAN results are sensitive to the precise value of ϵ , a grid search is conducted over a range of candidate values. Specifically, 12 evenly spaced ϵ values were tested in the interval from $0.4 \times \epsilon_{\text{elbow}}$ to $3.0 \times \epsilon_{\text{elbow}}$, corresponding to the range $[0.7006, 5.2544]$. For each tested combination of ϵ and MinPts: the number of clusters found, the percentage of noise points, and, where possible, the Silhouette Score on non-noise points were recorded. A parameter combination was considered acceptable if it produced between 2 and 10 clusters and between 5% and 40% noise. If one or more acceptable combinations had been found, the final choice would have been based on the highest silhouette score. If no acceptable combination existed, a fallback rule selected the configuration whose noise proportion was closest to 20%.

The DBSCAN parameter search did not reveal a stable multi-cluster solution in the normalized 50-dimensional segment space. For MinPts = 100, all tested ϵ values produced only a single cluster. For MinPts = 50, two clusters appeared only at the smallest tested ϵ , but this solution was associated with an extremely high noise proportion of roughly 80%, making it unsuitable as a meaningful clustering of the data. Across the remaining parameter range, the number of clusters quickly collapsed to one and the noise share approached zero. As a result, no parameter combination met the predefined criteria of producing both a reasonable number of clusters and an acceptable amount of noise. A detailed overview of the tested parameter combinations and their outcomes is provided in Appendix A (Table A1).

DBSCAN is implemented in `src/clustering/dbscan/dbscan.py` using the DBSCAN class from the scikit-learn library

6.4 t-SNE Visualisation

To visualise the high-dimensional cluster structure in two dimensions, t-SNE is applied to the 50,000-segment sample to produce a two-dimensional embedding of the 50-dimensional feature space.

To ensure that visual differences between algorithm plots reflect only label differences not coordinate randomness from independent t-SNE runs, the t-SNE projection is computed once and the resulting 2D

coordinates are saved for reuse. The same coordinates are then rendered three times, once colored by K-Means labels, once by Hierarchical labels, and once by DBSCAN labels, producing a side-by-side comparison that allows the spatial coherence of each partition to be assessed visually.

7 Results and Analysis

7.1 K-Means

The elbow method identified $k = 4$ as the point where the second derivative of inertia is maximized, suggesting that four clusters capture the principal modes of variation in the data. After running K-Means on the dataset, the algorithm converged in 37 iterations with a final inertia of 100,890.84 and a runtime of 0.4 seconds, producing a balanced partition of the 50,000 segments into the four clusters.

The cluster size distribution is relatively uniform: Cluster 1 is the largest with 14,548 segments (29.1%), followed by Cluster 3 with 14,225 segments (28.4%), Cluster 0 with 10,814 segments (21.6%), and Cluster 2 with 10,413 segments (20.8%), as shown in Figure 7:

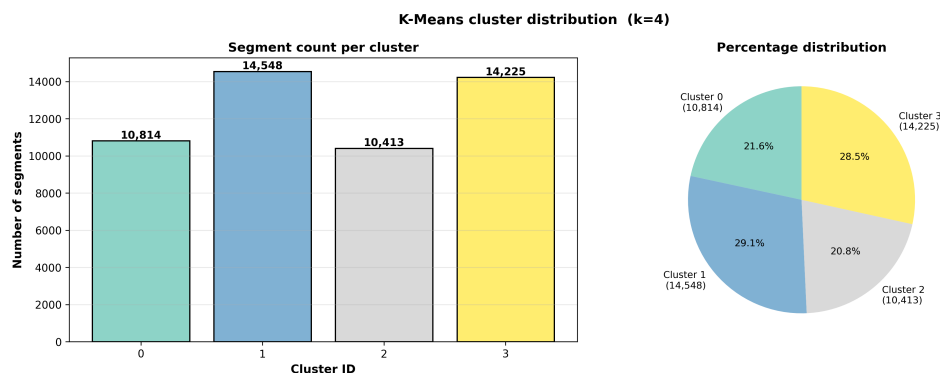


Figure 7: K-Means cluster distribution

7.2 Agglomerative Hierarchical Clustering

Hierarchical Clustering with Ward linkage required 459.8 seconds to construct the full dendrogram over the 50,000 segments. The dendrogram, shown in Figure 8, is cut at $k=4$ at a Ward distance of 132.25 resulting in four clusters:

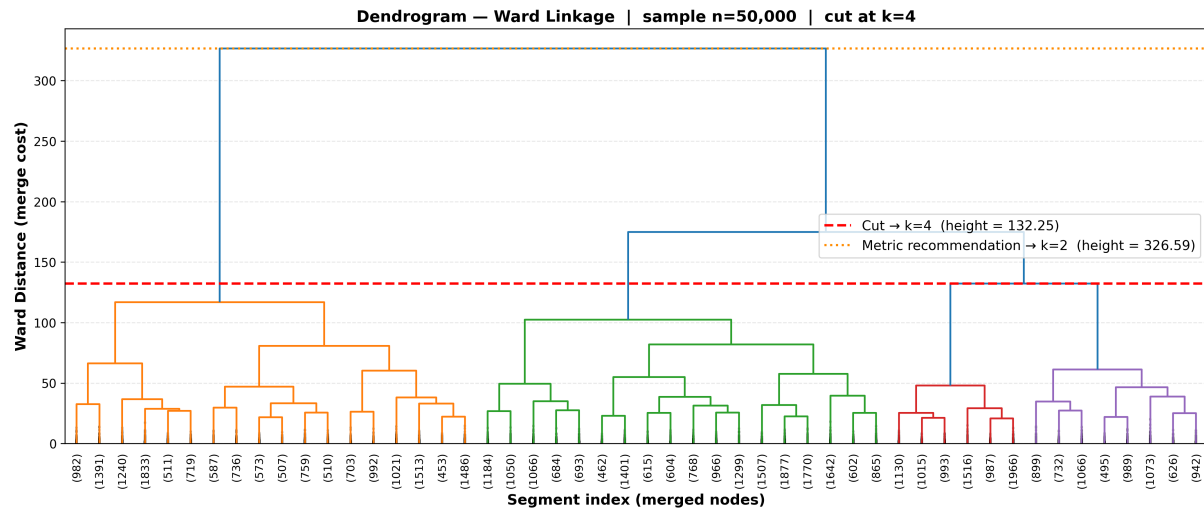


Figure 8: Hierarchical - Dendrogram

The cluster size distribution is notably less balanced than K-Means: clusters 0 and 1 together contain 71.1% of all segments, while clusters 2 and 3 together contain only 28.9%. This asymmetry is expected given that Ward linkage does not explicitly enforce size balance (which can also be seen in the dendrogram), unlike K-Means which implicitly tends toward more equal partitions through its centroid-based geometry.

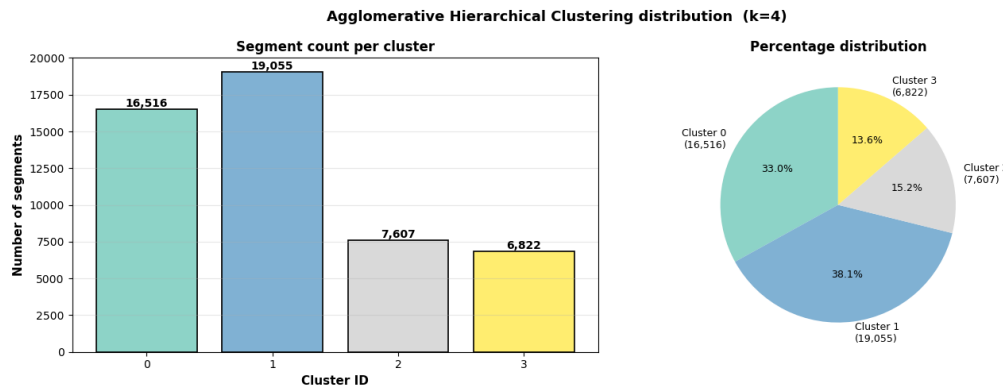


Figure 9: Hierarchical Clustering Cluster Distribution

7.3 Cluster Center Analysis

Examining the cluster centers, the mean normalized price trajectory of each cluster over the 50-day window, reveals four price pattern shapes in each of both algorithms, shown in Figure 9:

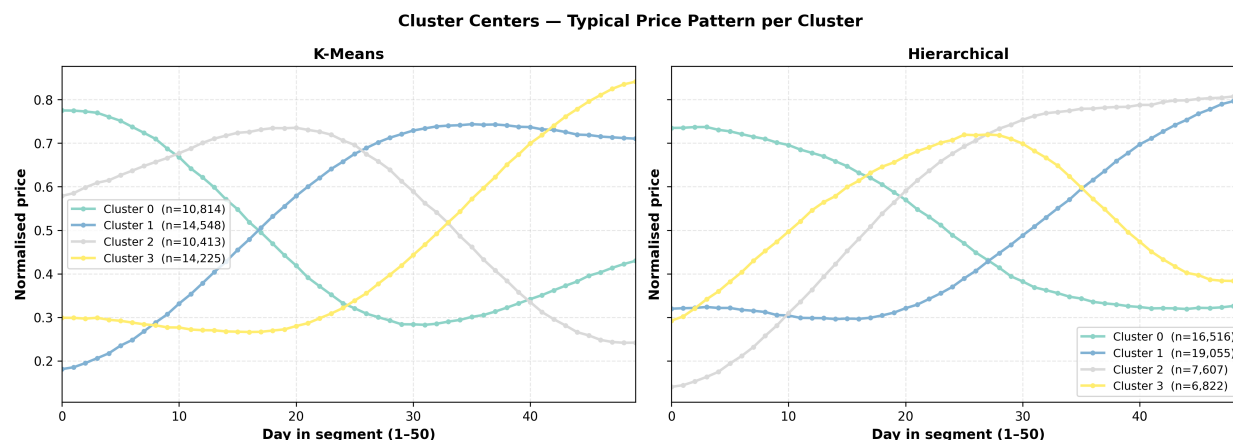


Figure 10: Cluster centers for K-Means and Hierarchical

A closer comparison of the cluster centers indicates that the K-Means and Agglomerative Hierarchical solutions recover broadly similar, though not identical, trajectory structures. The strongest correspondences are between **K-Means Cluster 3** and **Hierarchical Cluster 1**, both of which remain comparatively weak in the early part of the window before rising strongly later, and between **K-Means Cluster 1** and **Hierarchical Cluster 2**, which both represent generally increasing trajectories that level off toward the end. A further close match is given by **K-Means Cluster 0** and **Hierarchical Cluster 0**, as both start from a relatively high level and decline across the segment, although the K-Means center exhibits a more noticeable late rebound while the hierarchical center remains more nearly monotonic. The least exact, but still most plausible, correspondence is between **K-Means Cluster 2** and **Hierarchical Cluster 3**: both rise in the first half of the window, reach an interior maximum near the middle, and then decline. Nevertheless, they differ substantially in their starting level and overall net movement, since the K-Means center begins high and ends much lower, whereas the hierarchical center begins lower and still finishes above its starting point. Thus, the two methods recover the same broad pattern families, but differ in the precise location of turning points, the steepness of transitions, and the overall shape of the cluster prototypes.

The results are broadly related to Huang (2023), who also studies 50-day stock-price segments sampled at size 50,000 and reports that K-Means, DBSCAN, and hierarchical clustering yield very similar average patterns, which are interpreted as harmonic or cosine-like waves. However, Huang emphasizes a stronger cross-method similarity and a more explicitly harmonic interpretation, whereas the present thesis finds only moderate agreement between K-Means and hierarchical clustering.

Figure 10 represents Huang (2023) findings:

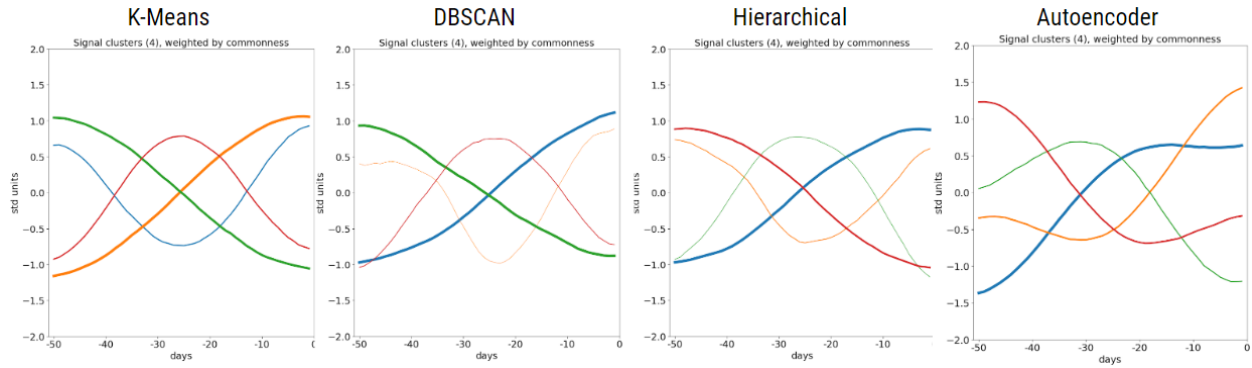


Figure 11: Average time series of clusters (Huang, 2023)

7.4 DBSCAN

As discussed in section 6.4, Since no combination met the acceptance criteria, the fallback decision rule identified $\text{eps} = 1.1146$ and $\text{MinPts} = 100$. Under this configuration, DBSCAN identified one cluster containing 41,224 segments (82.4%) and labeled 8,776 segments (17.6%) as noise. Since the final solution contains only one non-noise cluster, the Silhouette Score is not defined and is therefore not reported for the final DBSCAN partition.

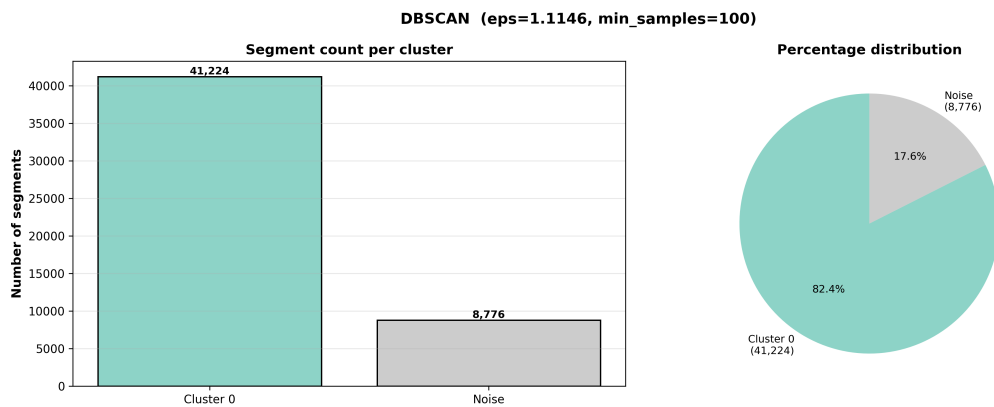


Figure 12: DBSCAN Clustering Distribution

this result suggests that the normalized stock-price segments do not form several clearly separated dense regions of the kind DBSCAN is designed to detect. Instead, the data appears to occupy one large, broadly connected density region with a moderate number of peripheral observations classified as noise. This does not imply that the data contains no structure at all. Rather, it suggests that any recurring structures present in the normalized stock-price segments are overlapping and weakly separated rather than sharply isolated in the feature space

7.5 Internal Validation Metrics

Figure 11 summarises the internal validation metrics for all three algorithms. For DBSCAN, which produced a single cluster, the metrics cannot be computed and are therefore not reported:

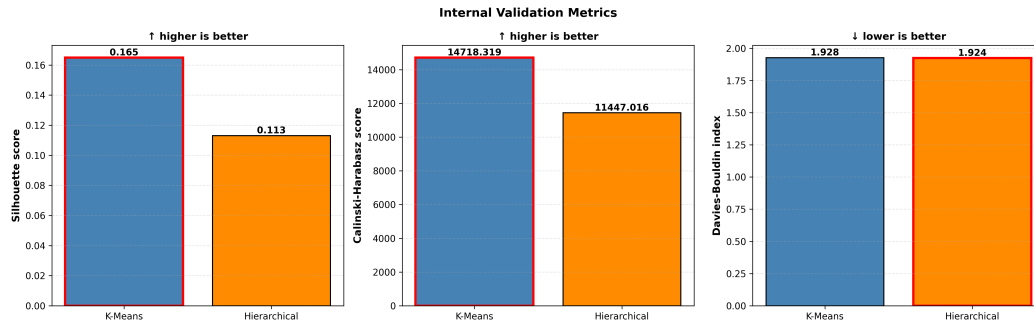


Figure 13: Internal validation metrics for all algorithms

K-Means outperforms Hierarchical Clustering on Silhouette Score (0.1650 vs. 0.1131) and Calinski-Harabasz Index (14,718.3 vs. 11,447.0), indicating more compact and better-separated clusters on average. The Davies-Bouldin Index is marginally better for Hierarchical Clustering (1.9243 vs. 1.9276), though the difference is negligible. Both algorithms produce Silhouette Scores below 0.25, classifying the cluster quality as poor.

7.6 Cross-Algorithm Consistency

Figure 14 and table 5 report the pairwise Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) between all three algorithms:

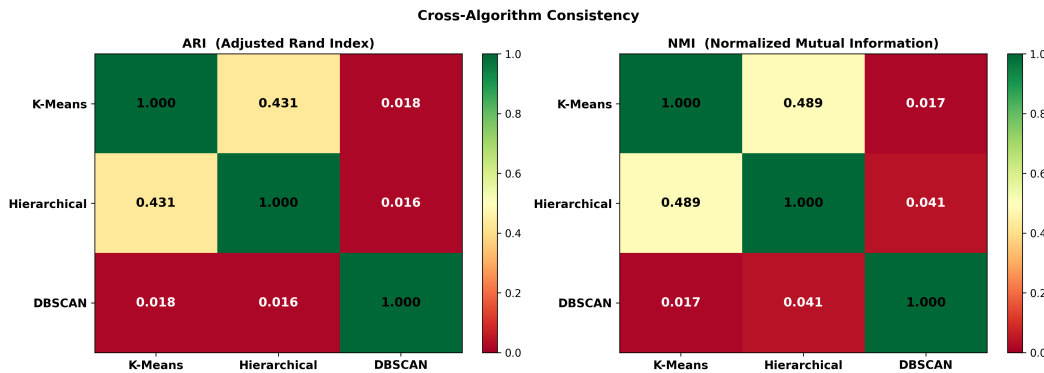


Figure 14 : Cross-algorithm consistency metrics

Pair	ARI	NMI	Interpretation
K-Means vs Hierarchical	0.431	0.489	Weak agreement
K-Means vs DBSCAN	0.018	0.017	No agreement
Hierarchical vs DBSCAN	0.016	0.041	No agreement

Table 5 : Cross-algorithm consistency metrics

The results show how similar the clustering outputs are when evaluated using both the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI), which capture agreement in different ways. The ARI of 0.431 and NMI of 0.489 between K-Means and Hierarchical Clustering indicate weak-to-moderate agreement. From the ARI perspective, this suggests that a fair proportion of point pairs are grouped consistently in both clusterings, but there are still many disagreements in how pairs are assigned to the same or different clusters. From the NMI perspective, the value indicates that the two clusterings share a moderate amount of information, meaning that knowing the cluster assignment from one method provides some insight into the other, though not reliably.

Since the ARI is corrected for chance, a value of 0.431 indicates that the agreement between K-Means and hierarchical clustering is substantially better than random assignment. Similarly, the NMI value of 0.489 suggests a moderate amount of shared information between the two clusterings, further confirming that both methods capture some common structure in the data, although the agreement remains far from perfect.

Both K-Means and Hierarchical Clustering showing near-zero agreement with DBSCAN is a direct consequence of DBSCAN finding only a single cluster. K-Means and Hierarchical Clustering assign every point to one of four clusters, while DBSCAN produced one cluster plus noise.

From the ARI perspective, K-Means and hierarchical clustering divide the data into multiple groups, meaning many point pairs are separated into different clusters by these two algorithms, while they are grouped together in a single cluster in DBSCAN. From the NMI perspective, DBSCAN provides very little information about the other clusterings, since having essentially one main cluster (plus noise) does not capture the more detailed grouping found by the other methods.

7.7 t-SNE Visualisation

The t-SNE projection of the 50,000-segment sample into two dimensions provides a visual perspective on the cluster structure:

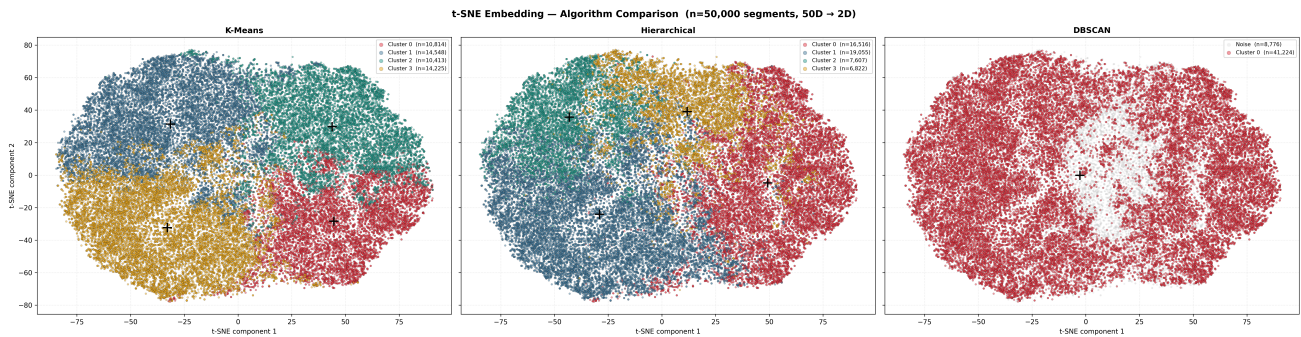


Figure 15: t-SNE Embedding - Algorithm Comparison

The 2D embedding reveals a single large, roughly continuous cloud of points with no sharply separated islands, consistent with the low Silhouette Scores observed across all algorithms and the failure of DBSCAN to find multiple density-separated clusters. K-Means produces more compact and separated clusters, while hierarchical clustering appears more uneven, with elongated shapes and greater mixing between clusters. This visual difference aligns with the internal metrics, where K-Means achieves higher Silhouette and Calinski-Harabasz scores, indicating better cohesion and separation. However, it is important to note that neither method achieves strong separation or compactness overall.

Both algorithms produce four regions with noticeable similarities in how data points are assigned to clusters, with several areas showing strong overlap in cluster membership. However, there is also a substantial number of points where the assignments and cluster boundaries differ between the two methods, reflecting inconsistencies in how the data is partitioned. This observation is consistent with the

results of the cross-algorithm evaluation in section 7.6, which indicated only moderate agreement between K-Means and hierarchical clustering. The DBSCAN plot shows almost entirely one colour with scattered gray noise points, reflecting the single-cluster result.

8 Discussion

8.1 Interpretation of Results

K-Means and Agglomerative Hierarchical Clustering both recover broadly comparable structures, and their agreement is clearly above chance. This indicates that the identified patterns are not purely arbitrary artifacts of a single algorithm. At the same time, the low silhouette scores and the only weak-to-moderate cross-method agreement show that these structures are not sharply defined. Rather than forming compact and clearly separated groups, the segment space appears to consist of overlapping trajectory forms with gradual transitions between them.

The ARI of 0.431 between K-Means and Hierarchical Clustering is substantially above zero, indicating agreement beyond what would be expected by chance, but still far below 1.0, showing that a substantial share of individual assignments differs between the two methods. The NMI value of 0.489 leads to the same overall interpretation: the two clusterings share a moderate amount of information, but not enough to be regarded as strongly consistent. This pattern is also reflected in the t-SNE visualisation, where both algorithms impose a four-region partition on the same broadly continuous cloud of points, but differ in how they divide the transition zones between trajectory families. K-Means, through centroid optimisation and its tendency toward more balanced partitions, produces more compact and evenly sized clusters. Hierarchical Clustering, through its merge structure and lack of explicit size constraints, produces a more asymmetric partition in which a larger share of points is concentrated in two dominant clusters.

The DBSCAN result adds an important geometric perspective that goes beyond the partition- and hierarchical-based methods. DBSCAN does not recover any stable multi-cluster configuration across the tested parameter combinations, and the parameter search yields no combination satisfying the predefined criteria for both cluster count and noise proportion. Although the k-distance graph produced an automatic elbow estimate, the broader search around this region does not reveal a robust density-separated solution. Instead, the final DBSCAN model identifies one large cluster together with a moderate amount of noise, suggesting that the normalized 50-dimensional segment space lacks the discrete high-density subregions that would be necessary for DBSCAN to detect multiple clusters.

This does not imply that the data contains no structure. Rather, it suggests that structures identified by K-Means and Hierarchical Clustering represent directional tendencies within a continuous and overlapping distribution rather than isolated populations separated by sparse regions. The t-SNE embedding supports this interpretation by showing a single large, roughly continuous cloud of points with no sharply separated islands.

8.2 Answering the Research Questions

This thesis set out to examine whether different unsupervised machine learning clustering algorithms can identify recurring price patterns in financial time series data, and to what extent the resulting structures are well-separated, compact, and consistent across methods. The results allow the three research questions to be answered as follows.

First, the results suggest that different unsupervised clustering algorithms can identify some recurring price patterns in the normalized 50-day stock-price segments, but not in a fully consistent way. K-Means and Agglomerative Hierarchical Clustering both recover broadly comparable smooth trajectory forms. Their cluster centers show several clear qualitative correspondences. However, these correspondences are only approximate rather than exact, and DBSCAN does not recover the same multi-cluster structure. Thus, recurring local price-pattern structure appears to be present, but its recovery depends on the clustering method used. Second, the resulting clusters exhibit only poor internal quality in terms of separation and compactness. K-Means performs better than Hierarchical Clustering on the Silhouette Score and Calinski–Harabasz Index, while the Davies–Bouldin values of the two methods are almost identical. However, both algorithms produce Silhouette Scores below 0.25, which indicates weak overall separation and compactness. This means that although K-Means yields the strongest partition among the tested methods, neither result can be regarded as strongly separated in an absolute sense. Third, the recovered structures are only moderately consistent across different clustering methods. The agreement between K-Means and Hierarchical Clustering reaches an ARI of 0.431 and an NMI of 0.489, indicating moderate agreement and therefore some shared non-random structure beyond purely arbitrary grouping. At the same time, this agreement remains far from perfect, showing that the two methods still partition the data differently in many cases. Both methods show near-zero agreement with DBSCAN, which is a direct consequence of DBSCAN identifying one single cluster together with noise rather than a multi-cluster structure.

Taken together, these answers suggest that the normalized stock-price segments do contain some recurring local structure, but that this structure is weakly separated, only moderately stable across methods, and not supported as a density-separated cluster configuration. The findings therefore support a cautious conclusion: the data appears to contain overlapping and continuous trajectory regularities rather than a small set of sharply distinct and universally recoverable pattern classes.

8.3 Limitations

Several limitations of this study should be acknowledged. First, the analysis uses only euclidean distance in the original normalized feature space. Although this choice ensures comparability across algorithms, it may not be the most appropriate similarity measure for financial time series. Second, the empirical design is based on fixed 50-day windows, a fixed stride of 5 trading days, and a single normalization scheme. Different window lengths, overlap levels, time periods, or normalization methods could reveal different forms of structure. The present findings should therefore be understood as conditional on this specific representation of the data. Third, the DBSCAN parameter search is not exhaustive, as only two MinPts values and 12 eps values are tested. A broader search across additional density thresholds would strengthen the robustness of the conclusion that the data does not contain a stable density-separated multi-cluster structure. Fourth, the t-SNE visualisation is informative but should be interpreted cautiously. As a non-linear dimensionality-reduction technique, t-SNE is useful for visual exploration, but it does not preserve all pairwise distances or the global structure of the original feature space faithfully. It therefore supports interpretation rather than providing conclusive geometric proof. Finally, the analysis is based on a random sample of 50,000 segments rather than the full dataset of 570,321 segments, and the findings may be affected to some extent by sampling variability.

8.4 Directions for Future Research

Several directions for future research follow from the findings of this thesis. First, future studies could examine whether the weak separation observed here reflects an inherent property of stock-price time series or instead results from the particular way the data was represented. This could be explored by testing alternative ways of measuring similarity between price segments and by using different representations of the same time series or by applying dimensionality reduction methods to reduce the

effective dimensionality of the feature space before clustering and potentially produce more compact and well-separated clusters. Second, the present study is based on fixed 50-day windows and one normalization method. Future work could investigate whether recurring structures become clearer when different window lengths, overlap settings, or preprocessing choices are used. It is possible that some forms of structure are more visible at shorter or longer horizons than in the specific segmentation scheme adopted here. Third, the comparison in this thesis is limited to three clustering algorithms, a broader comparison involving additional methods could help determine whether the present findings are stable across a wider range of clustering assumptions. Finally, this thesis focuses on structural discovery rather than practical usefulness, future work could therefore examine the economic relevance of the recovered structures by testing whether cluster membership is associated with systematic differences in subsequent returns, volatility, or regime behavior. Testing such relationships would help clarify whether the identified patterns are only descriptively interesting or also meaningful for downstream financial applications.

9 Conclusion

This thesis applied three unsupervised clustering algorithms, K-Means, Agglomerative Hierarchical Clustering with Ward linkage, and DBSCAN, to a shared sample of 50,000 normalized 50-day price segments extracted from S&P 500 constituent stocks over the period 2000–2025. The analysis was guided by three research questions concerning the identifiability of recurring patterns, the internal quality of the resulting clusters, and the consistency of the recovered structures across different methods. The findings suggest that the data contains some recurring local structure. K-Means and Hierarchical Clustering recover broadly comparable trajectory forms, and their cluster centers indicate similar, though not identical, pattern families. At the same time, the agreement between the two solutions is only moderate, and both methods produce weak internal validation results. K-Means achieves a Silhouette Score of 0.165, compared with 0.113 for Hierarchical Clustering, while their cross-method agreement reaches an Adjusted Rand Index of 0.431 and a Normalized Mutual Information of 0.489. DBSCAN, by contrast, does not identify a density-separated multi-cluster configuration under any of the tested parameter combinations, suggesting that the recovered trajectory families are better understood as directional tendencies within a continuous distribution rather than as discrete isolated populations.

The methodological contribution of the thesis lies in its systematic three-algorithm comparison supported by formal internal validation and cross-algorithm consistency measures, thereby providing a more rigorous evaluation framework than prior studies that rely mainly on a single method or do not assess the resulting structures after clustering. Future work could extend these findings by testing alternative algorithms, similarity measures and data representations and by examining whether the recovered clusters carry economic significance in terms of subsequent returns, volatility or market regime behavior.

Declaration of AI Use

Product name	URL
ChatGPT	https://chat.openai.com
Claude	https://claude.ai/

Type of support	Tool used	Description of use
Support in improving the language of the text	ChatGPT and Claude	Used to improve the language of individual paragraphs and passages (grammar, spelling, and style).
Support with programming / code development	Claude	Claude is used to assist with individual code sections, debugging, and clarifying technical questions. The generated code was not adopted without review; it was independently checked, adapted, tested, and integrated by into the overall project. The conceptual implementation, methodological choices, and final design are my own.

References

- Aghabozorgi, S., Seyed Shirkhorshidi, A., & Ying Wah, T. (2015). Time-series clustering – A decade review. *Information Systems*, 53, 16–38. <https://doi.org/10.1016/j.is.2015.04.007>
- Ashabi, A., Sahibuddin, S. B., & Salkhordeh Haghighi, M. (2021). The Systematic Review of K-Means Clustering Algorithm. *Proceedings of the 2020 9th International Conference on Networks, Communication and Computing, ICNCC '20*, 13–18. <https://doi.org/10.1145/3447654.3447657>
- Aslam, B. (2025). Identifying optimistic stocks with K-means clustering algorithm. *International Review of Economics & Finance*, 104, 104579. <https://doi.org/10.1016/j.iref.2025.104579>
- Barradas, A., Canton-Croda, R.-M., & Gibaja-Romero, D.-E. (2023). Identification of Patterns in the Stock Market through Unsupervised Algorithms. *Analytics*, 2(3), 592–603. <https://doi.org/10.3390/analytics2030033>
- Bholowalia, P., & Kumar, A. (2014). EBK-Means: A Clustering Technique based on Elbow Method and K-Means in WSN. *International Journal of Computer Applications*, 105(9).
- Bouguettaya, A., Yu, Q., Liu, X., Zhou, X., & Song, A. (2015). Efficient agglomerative hierarchical clustering. *Expert Systems with Applications*, 42(5), 2785–2797. <https://doi.org/10.1016/j.eswa.2014.09.054>
- Caliński, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3(1), 1–27. <https://doi.org/10.1080/03610927408827101>
- Chicco, D., Campagner, A., Spagnolo, A., Ciucci, D., & Jurman, G. (2025). The Silhouette coefficient and the Davies-Bouldin index are more informative than Dunn index, Calinski-Harabasz index, Shannon entropy, and Gap statistic for unsupervised clustering internal evaluation of two convex clusters. *PeerJ Computer Science*, 11, e3309. <https://doi.org/10.7717/peerj-cs.3309>
- Davies, D. L., & Bouldin, D. W. (1979). A Cluster Separation Measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224–227. <https://doi.org/10.1109/TPAMI.1979.4766909>

- Deng, D. (2020). DBSCAN Clustering Algorithm Based on Density. *2020 7th International Forum on Electrical Engineering and Automation (IFEEA)*, 949–953.
<https://doi.org/10.1109/IFEEA51475.2020.00199>
- Ester, M., Kriegel, H.-P., & Xu, X. (1996). *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*.
- Fu, T.-C., Chung, F.-L., Ng, V., & Luk, R. (2001). Pattern discovery from stock time series using self-organizing maps. *Workshop Notes of KDD2001 Workshop on Temporal Data Mining*.
- Gour, N. (2023, November 5). DBSCAN Algorithm. *Medium*.
<https://medium.com/@nitingour1203/dbscan-algorithm-f14e634abb02>
- He, Z. (2025). *Unsupervised Learning in Finance: From Bibliometric Analysis to Investment Applications* [Doctoral, Durham University]. <https://etheses.durham.ac.uk/id/eprint/16140/>
- Hennig, C., Meila, M., Murtagh, F., & Rocci, R. (Eds.). (2016). *Handbook of cluster analysis*. CRC Press, Taylor & Francis Group.
- Huang, L. (2023). *Searching for Patterns in Daily Stock Data: First Steps Towards Data-Driven Technical Analysis*. <https://www.jpmorgan.com/technology/technology-blog/searching-for-patterns>
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218.
<https://doi.org/10.1007/BF01908075>
- Javed, A., Lee, B. S., & Rizzo, D. M. (2020). A benchmark study on time series clustering. *Machine Learning with Applications, 1*, 100001. <https://doi.org/10.1016/j.mlwa.2020.100001>
- Nanda, S. R., Mahanty, B., & Tiwari, M. K. (2010). Clustering Indian stock market data for portfolio management. *Expert Systems with Applications*, 37(12), 8793–8798.
<https://doi.org/10.1016/j.eswa.2010.06.026>
- Pereira, C., & Mello, R. (2013). Common Dissimilarity Measures are Inappropriate for Time Series Clustering. *Revista de Informática Teórica e Aplicada*, 20, 25. <https://doi.org/10.22456/2175-2745.25070>

- Rand, W. M. (1971). *Objective Criteria for the Evaluation of Clustering Methods: Journal of the American Statistical Association: Vol 66, No 336.*
<https://www.tandfonline.com/doi/abs/10.1080/01621459.1971.10482356>
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
[https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (1998). Density-Based Clustering in Spatial Databases: The Algorithm GDBSCAN and Its Applications. *Data Mining and Knowledge Discovery*, 2(2), 169–194. <https://doi.org/10.1023/A:1009745219419>
- Shirkhorshidi, A. S., Aghabozorgi, S., Wah, T., & Herawan, T. (2014). Big Data Clustering: A Review. In *LNCS* (Vol. 8583). https://doi.org/10.1007/978-3-319-09156-3_49
- Strehl, A., & Ghosh, J. (2002). *Cluster Ensembles – A Knowledge Reuse Framework for Combining Multiple Partitions.*
- Vijaya, Sharma, S., & Batra, N. (2019). Comparative Study of Single Linkage, Complete Linkage, and Ward Method of Agglomerative Clustering. *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon)*, 568–573.
<https://doi.org/10.1109/COMITCon.2019.8862232>
- Warren Liao, T. (2005). Clustering of time series data—A survey. *Pattern Recognition*, 38(11), 1857–1874. <https://doi.org/10.1016/j.patcog.2005.01.025>
- What is Silhouette Score.* (2026, March 30). GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/what-is-silhouette-score/> [Accessed: Apr. 4, 2026]
- Winland, V., & Kavlakoglu, E. (2024, June 26). *What is k-means clustering? | IBM.*
<https://www.ibm.com/think/topics/k-means-clustering>
- Yang, Y., Lian, B., Li, L., Chen, C., & Li, P. (2014). *DBSCAN Clustering Algorithm Applied to Identify Suspicious Financial Transactions.* 60–65. <https://doi.org/10.1109/CyberC.2014.89>

Zhao, W.-L., Deng, C.-H., & Ngo, C.-W. (2018). *k*-means: A revisit. *Neurocomputing*, 291, 195–206.

<https://doi.org/10.1016/j.neucom.2018.02.072>

Appendix A

eps	min_samples	n_clusters	noise_pct	silhouette
0.7005823203101783	50	2	80.554	0.6436799764633179
1.1145627823116473	50	1	13.946	
1.5285432443131164	50	1	46	
1.942523706314585	50	1	0.0	
2.356504168316054	50	1	0.0	
2.7704846303175232	50	1	0.0	
3.184465092318992	50	1	0.0	
3.598445554320461	50	1	0.0	
4.01242601632193	50	1	0.0	
4.4264064783234	50	1	0.0	
4.840386940324868	50	1	0.0	
5.254367402326337	50	1	0.0	
0.7005823203101783	100	1	85.31400000000001	
1.1145627823116473	100	1	17.552	
1.5285432443131164	100	1	58	
1.942523706314585	100	1	0.0	
2.356504168316054	100	1	0.0	
2.7704846303175232	100	1	0.0	
3.184465092318992	100	1	0.0	
3.598445554320461	100	1	0.0	
4.01242601632193	100	1	0.0	
4.4264064783234	100	1	0.0	
4.840386940324868	100	1	0.0	
5.254367402326337	100	1	0.0	

Table A1: DBSCAN Parameter Search

GitHub repository: The source code used in this thesis for data collection, preprocessing, segmentation, normalization, clustering, evaluation, and visualization is available at:

<https://github.com/haamzaoui/financial-time-series-clustering.git>

A brief guide on how to reproduce the results is provided in `DOCUMENTATION.md`